{garias,cwilliams}@utec.edu.pe
CS4055: Networks and communications
W09

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

LAB3 TRANSPORT LAYER

TCP

The present evaluation will be received **only if the checkpoints of all the questions in the LABORATORY SESSION have been completed** with the course teacher.

For the checkpoints presentation, **have a doc or text file** where you present the answer to the questions (**including screenshots if necessary**). **Do not forget to save your tracefile.**

I. The submission must consist of:
   A. **Introduction** (include introductory paragraph):
   B. **Theoretical Framework** (definitions with references).
   C. **State of the Art** (research papers, articles, theses, pages with references).
   D. **Development of the experience** (including discussion and correct figure formatting)
   E. **Conclusions and recommendations** (at least one for each experience carried out).
   F. **References** (IEEE format)

II. The submission must be presented in **LaTeX** or Docs format (**pdf file**).

III. **Upload only the PDF of the report to Canvas**, use a link to the repository, include step-by-step photos of each experience and answer checkpoints where relevant, plus codes if they have been developed.

IV. **Only one member of the group will upload the report**.

V. **Do not forget to list the names of all members**.

{garias,cwilliams}@utec.edu.pe
CS4055: Networks and communications
W09

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

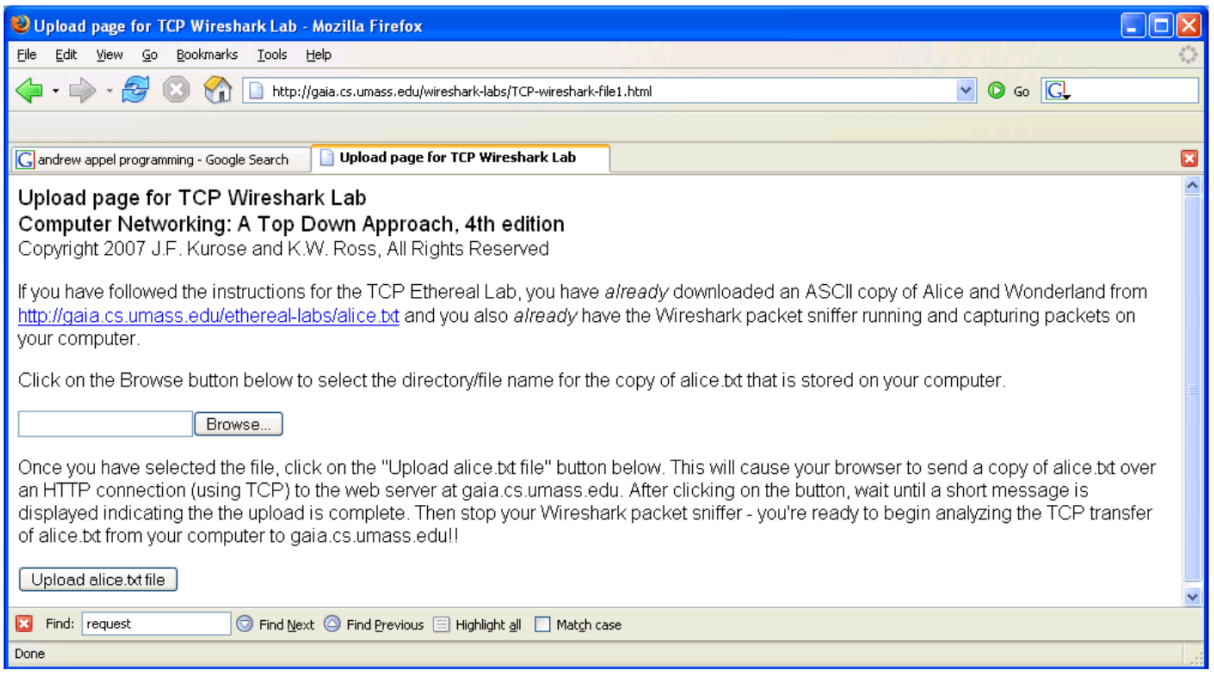
LAB3 TCP P1

1. Capturing a bulk TCP transfer from your computer to a remote server

Before beginning our exploration of TCP, we'll need to use Wireshark to obtain a packet trace of the TCP transfer of a file from your computer to a remote server. You'll do so by accessing a Web page that will allow you to enter the name of a file stored on your computer (which contains the ASCII text of Alice in Wonderland), and then transfer the file to a Web server using the HTTP POST. We're using the POST method rather than the GET method as we'd like to transfer a large amount of data from your computer to another computer. Of course, we'll be running Wireshark during this time to obtain the trace of the TCP segments sent and received from your computer.

Do the following:

- Start up your web browser. Go the http://gaia.cs.umass.edu/wiresharklabs/alice.txt and retrieve an ASCII copy of Alice in Wonderland. Store this file somewhere on your computer.
- Next go to
- http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html
- You should see a screen that looks like:



- Use the Browse button in this form to enter the name of the file (full path name) on your computer containing Alice in Wonderland (or do so manually). Don't yet press the "Upload alice.txt file" button.

- Now start up Wireshark and begin packet capture (Capture→Start) and then press OK on the Wireshark Packet Capture Options screen (we'll not need to select any options here).
- Returning to your browser, press the "Upload alice.txt file" button to upload the file to the gaia.cs.umass.edu server. Once the file has been uploaded, a short congratulations message will be displayed in your browser window.
- Stop Wireshark packet capture.

2. First look at the captured trace

2.1. Do the following:

- First, filter the packets displayed in the Wireshark window by entering "TCP" (lowercase, no quotes, and don't forget to press return after entering!) into the display filter specification window towards the top of the Wireshark window.
- What you should see is a series of TCP and HTTP messages between your computer and gaia.cs.umass.edu.
- You should see the initial three-way handshake containing a SYN message. You should see an HTTP POST message. Depending on the version of Wireshark you are using, you might see a series of "HTTP Continuation" messages being sent from your computer to gaia.cs.umass.edu.
- Recall from our discussion in the earlier HTTP Wireshark lab, that there is no such thing as an HTTP Continuation message – this is Wireshark's way of indicating that there are multiple TCP segments being used to carry a single HTTP message.
- In more recent versions of Wireshark, you'll see "[TCP segment of a reassembled PDU]" in the Info column of the Wireshark display to indicate that this TCP segment contained data that belonged to an upper layer protocol message (in our case here, HTTP). You should also see TCP ACK segments being returned from gaia.cs.umass.edu to your computer.
- Since this lab is about TCP rather than HTTP, let's change Wireshark's "listing of captured packets" window so that it shows information about the TCP segments containing the HTTP messages, rather than about the HTTP messages. To have Wireshark do this, select Analyze→Enabled Protocols. Then uncheck the HTTP box and select OK.

2.2. Questions:

Answer the following questions (Whenever possible,when answering a question you should hand in a printout of the packet(s) within the tracethat you used to answer the question asked. Annotate the printout to explain your answer. To print a packet, use File→Print, choose Selected packet only, choose Packet summary line, and select the minimum amount of packet detail that you need to answer the question).

1. **Identify the initial three-way handshake (provide a screenshot).**
2. What is the three-way handshake in TCP and why is it necessary before data transfer?
3. What role does the ACK play in TCP communication?
4. What specific information can be observed in the SYN, SYN-ACK, and ACK segments of the handshake?
5. What is the IP address and TCP port number used by the client computer (source) that is transferring the file to gaia.cs.umass.edu?
6. What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?
7. What happens in the capture when the HTTP protocol is disabled in Wireshark?
8. How many TCP segments were required to transfer the alice.txt file?
9. What patterns can be observed in the sequence numbers and acknowledgments (ACKs) during the transfer?

HINT.

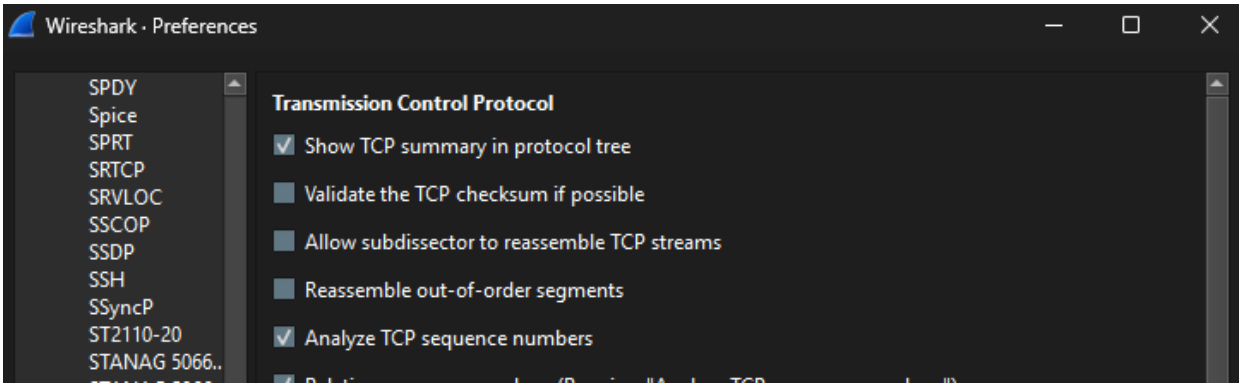
You can select a TCP packet in your trace file, right click it then go to **Follow>TCP Stream** to identify the flow of the selected communication.

CHECKPOINT 1

3. TCP Basics

For this section it is important that you **enable HTTP messages**. (To have Wireshark do this, select Analyze→Enabled Protocols. Then Check back on the HTTP box and select OK).
Also, let's deactivate the Reassembling:

Go to Edit > Preferences > Protocols > TCP: Now uncheck the: "Allow Subdissector to reassemble TCP Streams".



3.1. Questions:

Answer the following questions for the TCP segments:

1. What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu?
2. Where is the SYN segment?
3. What is the sequence number of the SYN ACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN?
4. What is the sequence number of the TCP segment containing the HTTP POST command? Note that in order to find the POST command, you'll need to dig into the packet content field at the bottom of the Wireshark window, looking for a segment with a "POST" within its DATA field.
5. Consider the TCP segment containing the HTTP POST as the first segment in the TCP connection. What are the sequence numbers of the first six segments in the TCP connection (including the segment containing the HTTP POST)? At what time was each segment sent? When was the ACK for each segment received? Given the difference between when each TCP segment was sent, and when its acknowledgement was received, what is the RTT value for each of the six segments? What is the EstimatedRTT value after the receipt of each ACK? Assume that the value of the EstimatedRTT is equal to the measured RTT for the first segment, and then is computed using the EstimatedRTT equation.

The first RTT would be the first ACK – Sent.

Start with the POST Message.

$$\text{EstimatedRTT}_n = 0.875 \cdot \text{EstimatedRTT}_{n-1} + 0.125 \cdot \text{SampleRTT}_n$$

	Sent time (s)	ACK Receiv time (s)	Sample RTT (s)	EstimatedRTT (Round-trip time)
Segment 1				
...				
...				
Segment 6				

6. What is the length of each of the first six TCP segments?

CHECKPOINT 2

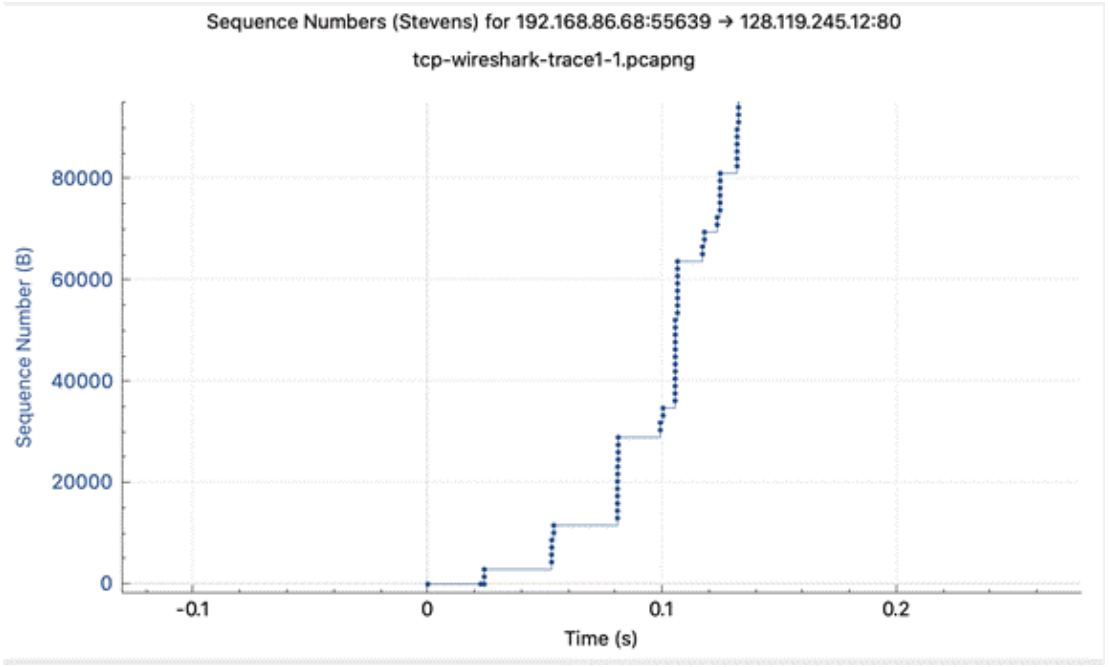
4. TCP congestion control in action

Let's now examine the amount of data sent per unit time from the client to the server. Rather than calculating this from the raw data in the Wireshark window, we'll use one of Wireshark's TCP graphing utilities–Time-Sequence-Graph(Stevens)–to plot out data.

- Select a client-sent TCP segment in the Wireshark's "listing of captured-packets" window corresponding to the transfer of alice.txt from the client to gaia.cs.umass.edu. Then select the menu: ***Statistics→TCP Stream Graph→ Time-Sequence-Graph(Stevens).***

Note. You may need to click the "Switch directions" option.

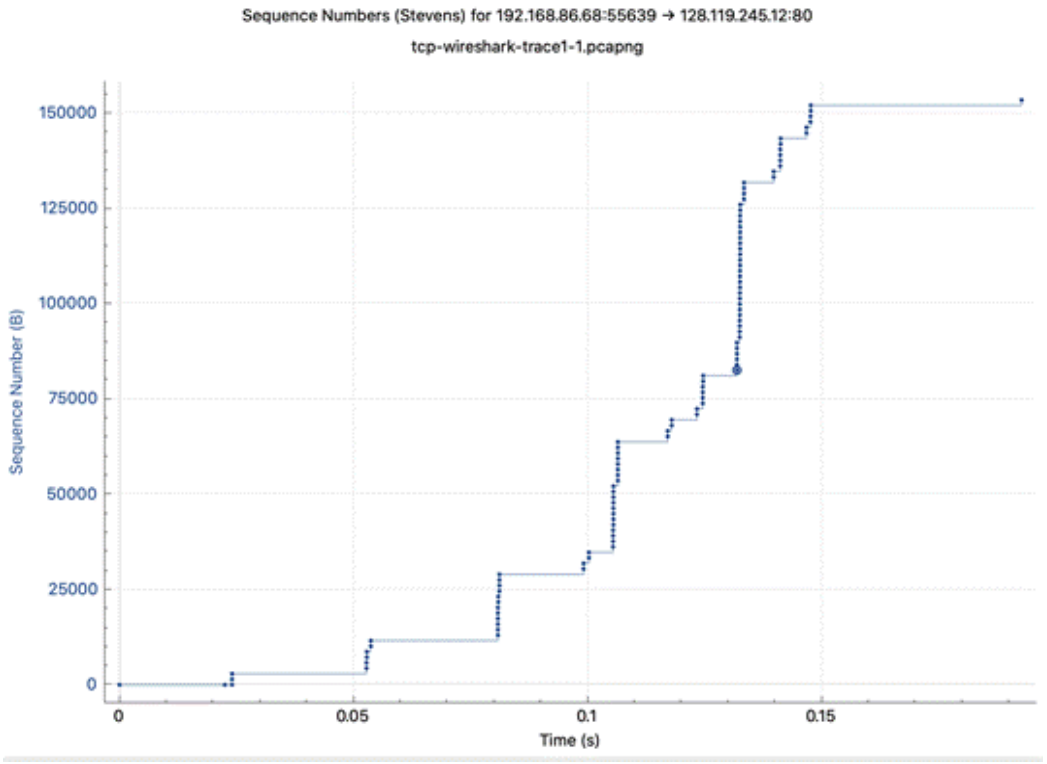
- You should see a plot that looks similar to the plot in figure below, which was created from the captured packets in the packet trace tcp-wireshark-trace1-1. You may have to expand, shrink, and fiddle around with the intervals shown in the axes in order to get your graph to look like the following figure.



Here, each dot represents a TCP segment sent, plotting the sequence number of the segment versus the time at which it was sent. Note that a set of dots stacked above each other represents a series of packets (sometimes called a "fleet" of packets) that were sent back-to-back by the sender.

4.1. Answer the following questions

1. Use the Time-Sequence-Graph(Stevens) plotting tool to view the sequence number versus time plot of segments being sent from the client to the gaia.cs.umass.edu server. Comment on whether this looks as if TCP is in its slow start phase, congestion avoidance phase or some other phase. Figure 6 shows a slightly different view of this data.
2. These "fleets" of segments appear to have some periodicity. What can you say about the period?
3. Answer each of two questions above for the trace that you have gathered when you transferred a file from your computer to gaia.cs.umass.edu



Another view of the same data.

CHECKPOINT 3